

Dual Push-Pull Activation Steering for Small Reasoning Models

Elham Bin Monsoom
Dept. of ECE
North South University
Dhaka, Bangladesh
ID: 2122299042

Nabila Yesmin
Dept. of ECE
North South University
Dhaka, Bangladesh
ID: 2212310042

Sk. Ilham Bin Rahman
Dept. of ECE
North South University
Dhaka, Bangladesh
ID: 2323045642

Abstract—When writing code, language models often suffer from overthinking loops or give into patterns that lead to coding mistakes. These are particularly pronounced in Small Language Models (SLMs) like VibeThinker-3B. Traditionally, fixing these tendencies requires retraining procedures that can be computationally expensive, or layers of prompt engineering that cause token and context bloat and affect the efficiency of the system. Inspired by Cognitive Behavioral Therapy (CBT), we propose a framework that intercepts the model’s internal activations at inference-time. CBT teaches individuals to recognize detrimental thought patterns, intervening in real time and steering away from them without complicated load-bearing external mechanisms or altering the core brain structure. Similarly, we isolate and subtract the “failure vectors” and redirect the model away while also pulling it towards clear, structured reasoning obtained from DeepSeek-R1’s powerful internal reasoning monologues. Our approach achieves 87% Pass@1 accuracy when evaluated across 100 HumanEval coding tasks that have historically caused subpar performance in SLM models. It outperformed negative prompt steering without suffering from context bloat, running in $O(1)$ real-time with zero added latency, thus offering a practical, superior and efficient path towards more reliable code generation by smaller language models.

I. INTRODUCTION

SLM models like VibeThinker-3B are capable of complex chain-of-thought reasoning and have been shown to match the performance of frontier LLMs despite their size. However, they are also prone to stability issues during code synthesis. The mistakes they make are not random and follow distinct patterns. In neuroscience, Magnetic Resonance Imaging (MRI) allows for the visualization of activity in the brain during certain processes or actions. Failures in tasks involve cognitive processes that have a geographical location in the brain. Similarly, we are able to probe VibeThinker-3B’s internal activations to determine the equivalent location in its hidden layers.

Standard preventative measures against such failures involve negative prompting or fine-tuning the model, which can be computationally expensive and cause generalizability to suffer since the model becomes better suited to the particular task it was fine-tuned for but worse at others. Negative prompting on the other hand, while effective leads to context bloat over time: the more it fails, the more it becomes necessary to remember what not to do. Thus, heavy external harnesses, memory management, prompt injection and pruning strategies

become necessary in order to cope with the rapidly expanding prompt bloat. CBT benefits people by identifying detrimental behavioral and thought patterns and training the individual’s mind to gradually move away from them while actively guiding towards more constructive alternatives.

In this work, we emulate CBT principles into a light steering framework through:

- **Layer Probing:** Identifying the critical geographical areas in the “brain” of the model responsible for code syntax and correctness, similar to MRI mapping.
- **Dual Push-Pull Vector:** Simultaneously pushing the model’s cognitive processes away from detrimental patterns and towards a more constructive and encouraged standard provided by the reasoning traces of DeepSeek-R1.
- **Zero Overhead:** Our intervention is executed without complex external systems or expensive training procedures and functions as part of the “brain” once incorporated.

II. BACKGROUND WORK / LITERATURE REVIEW

Our methodology is based on foundational representation engineering and activation addition research.

- **Representation Engineering (Zou et al., 2023):** Established maximum-margin support vector classifiers to extract contrastive activation vectors from model residual streams and demonstrated functional layer decomposition.
- **Activation Addition (Turner et al., 2023):** Provided the theoretical basis for forward hook activation steering scaled relative to empirical activation norm variance ($\alpha^* = 0.08$).
- **Signal-to-Noise Gating (Rimsky et al., 2024):** Introduced dynamic cosine similarity thresholding ($\tau^* = 0.15$) derived from $+2\sigma$ upper noise bound rules to prevent over-steering on clean decoding steps.
- **Reasoning Distillation (DeepSeek-AI, 2025):** Inspired positive cognitive pattern extraction ($\vec{v}_{\text{gist}}$) from distilled chain-of-thought trajectories under a subordinate attraction ratio ($\beta^* = 0.02$).

III. PROPOSED METHODOLOGY

Our framework involves two distinct phases: an offline vector extraction phase and an online inference phase where a single-layer forward hook applies to bring about real-time course correction away from failure spaces.

To that end, we constructed 8 contrastive prompt pairs synthesized from HumanEval benchmarking tasks. Each pair consists of two versions of the same programming task, one with a programming trap fallen into and the other with that trap avoided and overcome:

- 1) **Off-by-one indexing:** `range(len(arr) + 1)` vs. `range(len(arr))`
- 2) **Type mismatches:** `"Score: " + 10` vs. `"Score: " + str(10)`
- 3) **Uninitialized state:** Accumulating into uninitialized variable vs. `total = 0`
- 4) **Syntax errors:** Missing colons after loop and condition headers
- 5) **Recursion base cases:** Recursive calls lacking stopping guards
- 6) **Key lookups:** Direct unsafe indexing `d[key]` vs. `d.get(key, default)`

These pairs were passed through VibeThinker-3B and their internal hidden states across all 36 transformer layers were extracted. Our experiments revealed that the midpoint of the network, Layer 18 showed the sharpest linear separation between the prompt pair activations. Thus, we chose Layer 18 as our locus of modification. We used a Linear Support Vector Machine (SVM) on the L18 activations to obtain the hyperplane ($\mathbf{w} \cdot \mathbf{h} + b = 0$). Normalizing this decision boundary gives us our unit failure vector:

$$\hat{\mathbf{v}}_{\text{fail}} = \frac{\mathbf{w}}{\|\mathbf{w}\|_2} \in \mathbb{S}^{2047} \quad (1)$$

The repulsion vector is only half the process, since it only tells the model what not to do. For constructive improvement, we extracted the positive reasoning direction from DeepSeek-R1 Chain-of-Thought (CoT) traces occurring between the `<think>...</think>` tags. These were fed into our model, and the average activation at Layer 18 was recorded as $\mathbf{v}_{\text{gist}}^{\text{raw}}$.

In other words, we recorded the neural activation in our model upon seeing DeepSeek-R1 solve a particular problem, after which our goal was to pull our model towards patterns and behaviors that would recreate that exact signal. We applied *Gram-Schmidt Orthogonalization* to make it strictly perpendicular to the failure vector to ensure that pulling the model toward structured reasoning does not end up pushing it back towards the failure space:

$$\hat{\mathbf{v}}_{\text{gist}} = \text{Normalize}(\mathbf{v}_{\text{gist}}^{\text{raw}} - (\mathbf{v}_{\text{gist}}^{\text{raw}} \cdot \hat{\mathbf{v}}_{\text{fail}}) \hat{\mathbf{v}}_{\text{fail}}) \quad (2)$$

We executed the following update at every token step t , intercepting the hidden state in L18:

$$h_t'^{(18)} = h_t^{(18)} - \underbrace{\alpha \cdot \mathbb{I}(S_{t,\text{fail}} > \tau) \hat{\mathbf{v}}_{\text{fail}}}_{\text{Failure Repulsion Push}} + \underbrace{\beta \cdot \mathbb{I}(t \leq 200) \hat{\mathbf{v}}_{\text{gist}}}_{\text{Reasoning Attraction Pull}} \quad (3)$$

$$S_{t,\text{fail}} = \left\langle \frac{\mathbf{h}_t}{\|\mathbf{h}_t\|_2}, \hat{\mathbf{v}}_{\text{fail}} \right\rangle \quad (4)$$

This represents the real-time cosine alignment between the model's current thoughts and the failure direction. We derived the gated threshold ($\tau = 0.04$) following the representation geometry of Marks & Tegmark (2023). For VibeThinker-3B with hidden dimension 2048, the standard deviation of random background noise on the hypersphere is:

$$\sigma = \frac{1}{\sqrt{2048}} \approx 0.0221 \quad (5)$$

We wanted the failure repulsion to only be applied when the model's alignment with the failure direction is statistically significant with $p < 0.05$; thus, we set our gating threshold at two standard deviations:

$$\tau = 2\sigma = 2(0.0221) = 0.0442 \approx 0.04 \quad (6)$$

Therefore, this ensures that the intervention is silent when the code is generating just fine and triggers only when the model begins to decisively enter failure space.

- $\alpha = 0.08$ (Repulsion Strength): The optimal push intensity found through hyperparameter sweeps.
- $\beta = 0.015$ (Attraction Strength): To create a gentle pull only during early reasoning to establish structure before final code is emitted.

IV. EXPERIMENTS

We evaluated our framework on 100 coding tasks taken from the official OpenAI HumanEval benchmark. These tasks were selected for historically causing high crash rates or suboptimal performance (high latency, thinking loops, etc.) in earlier rounds of our experimentation. VibeThinker-3B naturally performs quite well on the benchmarks (what we called the Vanilla metric, i.e., baseline performance with no intervention). Thus, the failures for which intervention would be necessary were few and far between. Focusing on the failure-intensive set allowed us to see stutters in the vanilla performance more prominently, with the most likely to fail tasks clustered together.

All experiments used greedy argmax decoding ($T = 0.0$) with a token budget of 1024 tokens. Setting the temperature to 0.0 ensures that the runs are 100% deterministic and reproducible, and not due to random sampling noise.

We then conducted a 5-way comparative testing suite across all 100 selected failure-intensive tasks ($100 \times 5 = 500$ total evaluations):

- 1) **Vanilla Baseline:** The default, unsteered VibeThinker-3B model.

- 2) **Negative System Prompting:** Adding text guardrails to the prompt window.
- 3) **Static Failure Steering:** Constantly applying the failure repulsion push of 0.08 on every token.
- 4) **Adaptive RepGated Steering:** Conditionally applying repulsion only when the failure alignment exceeds the noise gate threshold.
- 5) **Dual Push-Pull DeepSeek:** Simultaneously applying failure repulsion and DeepSeek reasoning attraction.

To evaluate both binary correctness and continuous internal model dynamics, we tracked:

- Pass@1 Accuracy
- 95% Wilson Score Intervals
- McNemar’s Paired Exact Test
- Shannon Information Entropy ($H(p_t)$): Measuring predictive uncertainty across logits:

$$H(p_t) = - \sum_{v \in V} p_t(v) \log_2 p_t(v) \quad (7)$$

- 3-Gram Repetition Ratio (R_{rep}): Measures the fraction of repetitive text in the output:

$$R_{\text{rep}} = 1.0 - \frac{|\text{Unique 3-Grams}|}{\text{Total 3-Grams}} \quad (8)$$

Where a value of 0.0 is ideal, indicating that there are no loops and that every 3-token sequence is unique. A value of 1.0 indicates the model is caught in a loop and is repeating the same sequences over and over.

V. RESULTS

Our Dual Push-Pull Steering emerged as the top performer in the 5-way comparative benchmark across the 100 selected failure-intensive HumanEval tasks, achieving 87.00% Pass@1 accuracy. It outperformed negative prompting, static failure steering, and adaptive rep-gating, while maintaining stable predictive entropy and generation speed. The complete quantitative results are presented in Table I.

TABLE I: Benchmark Performance on 100 HumanEval Tasks

Variant	Pass@1 (%)	95% CI	Entropy $H(X)$	tok/s
1. Vanilla Baseline	84.00%	[75.6%, 89.9%]	0.485	27.83
2. Negative Prompting	84.00%	[75.6%, 89.9%]	0.569	27.56
3. Static ($\alpha = 0.08$)	85.00%	[76.7%, 90.7%]	0.487	27.48
4. RepGated ($\tau = 0.04$)	81.00%	[72.2%, 87.5%]	0.486	27.54
5. Dual DeepSeek	87.00%	[79.0%, 92.2%]	0.491	27.48

VI. ANALYSIS

Our results revealed that Negative prompting broke 13 problems that the Vanilla baseline solved, while also fixing 13 that Vanilla failed. Combined with the +17.3% entropy increase, it provides evidence that the negative constraint injections sometimes end up distracting the model away from its ideal performance, causing failures where it did not even need intervention. In contrast, our approach operated silently in the background without surges in entropy and distracting effects to the model.

The comparison between static steering and our approach highlights the latter’s utility. Static steering acts as a guardrail. It repels the model away but there is no determined direction for it go, which means every so often it goes off careening in a direction in the vector space that it cannot recover from via a logical path. The addition of the orthogonalized DeepSeek pulling vector acts as a lighthouse in the unknown, directing the model towards a particular direction and thereby enabling the model to achieve the lowest 3-gram repetition ratio amongst all the steered variants, preventing overthinking loops.

Analyzing the token trajectories of the Adaptive Gated Steering revealed the model flickering over and under the set threshold suggesting that the set value was too low. This resulted in the task performance suffering from noise.

VII. CONCLUSION

Our interventions and testing proves empirically that intercepting activations at L18 mitigates the instability that SLM suffer from, and that active push-pull steering directs model performance improvements more reliably than negative prompting.

REFERENCES

- [1] Comprehensive Technical Lab Report: Dual Push-Pull Activation Steering for Small Reasoning Models, WeiboAI Research, August 2026.
- [2] A. Zou et al., “Representation Engineering: A Top-Down Approach to AI Transparency and Control,” arXiv:2310.01405, 2023.
- [3] A. M. Turner et al., “Steering Language Models Without Optimization,” arXiv:2308.10248, 2023.
- [4] N. Rinsky et al., “Steering Llama 2 via Reasoning Vector Addition,” Alignment Research Center, 2024.
- [5] DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” arXiv:2501.12948, 2025.
- [6] WeiboAI Team, “VibeThinker-3B Model Card and System Technical Report,” arXiv:2606.16140, 2026.